

Firestore Development Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 1.5 Months

Level: Beginner

Course page: <https://archerinfotech.in/courses/database-technologies/firebase-training-in-pune>

Course Overview

Firestore is the de-facto Backend-as-a-Service (BaaS) for mobile-first applications in Pune startups — single-team mobile / web teams, solopreneurs, and early-stage fintech / consumer-tech companies use Firestore to ship full-stack mobile apps without building or operating a backend. Archer Infotech's Firestore Development training in Pune teaches the platform as it is actually used in 2026 — the modern Firestore stack on Google Cloud (post the 2024 'Firestore is now part of Google Cloud' branding rollout), Firestore for real-time document database, Firestore Authentication, Cloud Functions (now powered by Cloud Run under the hood), Firestore Hosting, Firestore Cloud Messaging, plus the newer additions (Firestore Genkit for GenAI, Firestore Data Connect for SQL-relational integration, Firestore App Hosting). The course is the right complement to our Android / Flutter / React Native / React courses for students who want to build full-stack mobile apps with one team. Classroom in Kothrud, online live, and weekend batches available.

Why Learn Firestore in 2026

Firestore is the most-deployed mobile-app backend in Pune startup hiring — Indeed Pune doesn't list 'Firestore Developer' as a standalone job title (it's not), but Firestore fluency is listed as required or preferred on 800+ Pune mobile / full-stack JDs as of May 2026. The biggest employers using Firestore as production backend are Pune-based fintech / consumer-tech / health-tech / ed-tech startups, plus several Pune captives and BFSI mobile teams that use Firestore for specific apps. The economics are compelling — for early-stage and mid-stage companies, Firestore replaces a backend team with a managed service.

What changed in 2026: Firestore has fully integrated into Google Cloud (the 2024 rollout). Cloud Functions for Firestore is now Cloud Run-backed under the hood. Firestore Genkit (released 2024) is the GenAI toolkit for building AI-powered features on Firestore. Firestore Data Connect (released 2024) brings managed Postgres into the Firestore ecosystem for the use cases where Firestore's document model isn't right. Firestore App Hosting (preview / GA in 2024–2025) is the modern deployment path for Next.js / Angular / Vue web apps with SSR support. Firestore has gotten Realtime Database-equivalent latency, plus better aggregations and indexing.

What this means for hiring: 2026 Pune Firestore JDs (or rather, mobile / full-stack JDs that list Firestore) expect Firestore data-modelling at depth (the security-rules-driven access model is unique), Firestore Auth flows, Cloud Functions for serverless logic, FCM for push notifications, plus the newer additions where they earn their place (Genkit for GenAI, Data Connect for SQL).

- Listed as required / preferred on 800+ Pune mobile / full-stack JDs (May 2026)
- De-facto BaaS for Pune startup mobile apps
- Firestore fully integrated into Google Cloud (2024 rollout)
- Firestore + Firestore Auth + Cloud Functions + FCM — the core 2026 stack
- Genkit for GenAI + Data Connect for SQL — 2024+ additions

Who This Course Is For

This course fits you if

- Working Android / iOS / Flutter / React Native developer wanting to add Firebase for full-stack mobile
- Working React / Next.js developer wanting Firebase for serverless backend
- Engineering / BCS / MCA student preparing for our mobile tracks (Firebase is a natural complement)
- Founder / solopreneur shipping a mobile MVP — Firebase is the right backend for solo / small teams
- Working backend developer wanting to add a managed BaaS to your toolkit
- Career restarter targeting Pune startup mobile-development

This course is not the right fit if

- If your goal is heavy-OLTP transactional workloads (financial-ledger style) — Firestore isn't the right tool
- If your goal is enterprise / BFSI backend — Firebase isn't the right institutional fit
- If you want SQL / relational depth — pick our Postgres or Oracle course
- If you cannot put in 5–6 hours per week of practice outside class
- If you have 2+ years of production Firebase experience — talk to us about advanced specialisations

Detailed Curriculum

Module 1: Firebase in 2026 — Platform, Projects & the Emulator Suite (Week 1)

What Firebase actually is after the Google Cloud consolidation: a set of managed services sitting on the same projects, IAM and billing as the rest of Google Cloud, rather than a separate product. The product map is worked through so that later choices are informed — Authentication, Firestore, Realtime Database, Cloud Functions, Cloud Storage, Hosting, Cloud Messaging, Crashlytics, Analytics, Remote Config — with the question each one answers.

The project is then set up the way a team would rather than the way a tutorial does: separate projects for development, staging and production, the Firebase CLI and project aliases, configuration kept out of source control, and the Local Emulator Suite running Auth, Firestore, Functions and Storage on the laptop. The emulator is treated as the daily driver from week one, because developing against a live production project is how student projects lose data and run up bills.

- Firebase as part of Google Cloud — projects, IAM and billing
- The product map and what each service is for
- Separate dev, staging and production projects
- The Firebase CLI, project aliases and firebase.json
- The Local Emulator Suite as the daily driver
- Seeding and exporting emulator data
- SDK setup for web, Android, iOS and Flutter
- Spark and Blaze plans, and what actually triggers cost

Module 2: Firebase Authentication (Week 1)

Identity, which every other Firebase feature depends on because security rules are written against it. The provider set is built out in class — email and password with verification and reset, phone and OTP as the dominant flow in India, Google, Apple, GitHub and Microsoft federated sign-in, and anonymous accounts later upgraded to a permanent identity without losing the user's data.

The token model is then made explicit: the ID token, its expiry and refresh, and how it reaches both security rules and your own backend. Custom claims are used for role-based access, with the propagation delay that catches people out; the Admin SDK verifies tokens server-side; and custom tokens integrate an existing authentication system. Multi-factor authentication, account linking, and the session and sign-out edge cases close the module.

- Email and password, verification and password reset
- Phone and OTP sign-in, and its India-specific rate limits
- Google, Apple, GitHub and Microsoft federated sign-in
- Anonymous accounts and upgrading them in place
- ID tokens, refresh and expiry

- Custom claims for roles, and claim propagation delay
- Admin SDK token verification on your own backend
- Custom tokens for an existing auth system
- Multi-factor authentication and account linking

Module 3: Firestore — Data Model, Queries & Aggregation (Week 2)

Firestore's model in its own terms: documents, collections and subcollections, document references, the 1MB document limit, and the field types that differ from plain JSON — timestamps, geopoints, references and arrays.

Queries are then taught alongside their constraints, because Firestore's limits are the thing that most shapes a design. Equality and range filters and the single-range-field rule, orderBy interaction, array-contains and in with their element caps, cursor pagination with startAfter, and the composite indexes the console asks you to create. Real-time listeners are contrasted with one-off reads on both cost and behaviour, count, sum and average aggregation queries are used to avoid reading a collection to size it, and transactions and batched writes complete the module.

- Documents, collections, subcollections and references
- Field types — timestamp, geopoint, reference, array, map
- Equality and range filters, and the single-range rule
- orderBy, array-contains, in and their limits
- Cursor pagination with startAfter and limit
- Composite indexes and index exemptions
- Real-time listeners against one-off reads
- count, sum and average aggregation queries
- Transactions and batched writes

Module 4: Firestore Data Modelling for Scale (Week 2)

Modelling for a database billed per document read, where the cost of a design is visible on an invoice. Denormalisation is the default rather than a compromise: duplicate the fields a screen needs so the screen is one read, and keep the copies consistent with a Cloud Function. Root collections against subcollections, and the collection-group query that makes subcollections searchable across parents.

Counters and aggregates are held as fields rather than computed on read, with distributed counters for the write-rate ceiling of one sustained write per second per document. The write-hotspot problem is demonstrated — sequential document IDs concentrate writes on one index range — along with fan-out for feeds, the security-rules-driven access model that constrains schema shape, and pagination designs that stay cheap.

- Denormalising so a screen is a single read
- Root collections against subcollections
- Collection-group queries
- Keeping duplicated data consistent with Functions
- Counters, aggregate fields and distributed counters
- The one-write-per-second document ceiling
- Write hotspots from sequential document IDs
- Fan-out patterns for feeds and notifications
- Modelling around what security rules can express

Module 5: Security Rules (Week 3)

The module that decides whether an application is safe, and the one most Firebase tutorials skip. Firestore and Storage rules share a language and a mental model: rules are not filters, an unauthorised query fails rather than returning fewer rows, and a rule that reads another document costs a read and can be denied itself.

The work is hands-on and adversarial. Rules are written for ownership, roles from custom claims, field-level validation and immutable fields; get and exists lookups are used for membership checks with their cost made explicit; and the Rules Playground and emulator unit tests turn rules into something with a test suite. The class then attacks its own project from a plain client SDK with a stolen user token — the exercise that makes the difference between rules that look right and rules that hold.

- The rules language, and why rules are not filters
- Ownership rules with request.auth
- Roles from custom claims
- Field-level validation and immutable fields
- get and exists lookups, and what they cost
- Storage rules for uploads — type, size and path
- The Rules Playground and emulator unit tests
- Attacking your own project from a client SDK
- The common rule mistakes that leak whole collections

Module 6: Realtime Listeners, Offline Persistence & Sync (Week 3)

The behaviour that makes Firebase feel different to build on: an application that keeps working without a network. Offline persistence on mobile and in the browser, the local cache, latency compensation where a write appears instantly and reconciles later, and the pending-writes metadata a considered UI actually shows the user.

Conflict handling is worked through honestly — last-write-wins is the default, and transactions or server timestamps are what you use when that is not acceptable. Listener lifecycle and detachment are covered because leaked listeners are both a memory bug and a billing one. The Realtime Database is then compared with Firestore on query model, pricing and latency, with the cases where it is still the better choice, such as presence.

- Offline persistence on mobile and web
- The local cache and latency compensation
- Pending writes and metadata-aware UI
- Last-write-wins, and when it is not acceptable
- Server timestamps and transactional updates
- Listener lifecycle, detachment and leaks
- Realtime Database against Firestore
- Presence with the Realtime Database
- Testing offline behaviour in the emulator

Module 7: Cloud Functions for Firebase (Week 4)

Server-side logic without a server to operate. Second-generation Cloud Functions run on Cloud Run, and the trigger types are covered with the job each one suits: HTTPS and callable functions for client-invoked work, Firestore and Auth and Storage triggers for reacting to events, scheduled functions for recurring jobs, and Pub/Sub and Task Queue functions for background work.

The operational realities are given equal weight, because they are what breaks in production: cold starts and minimum instances, concurrency and instance limits, idempotency for a trigger that may fire twice, retries and dead-lettering, and the infinite loop created by a function that writes to the document that triggered it. Secrets, the Admin SDK's privileged access, structured logging, error reporting and local debugging in the emulator complete the module.

- Second-generation functions on Cloud Run
- HTTPS and callable functions
- Firestore, Auth and Storage event triggers
- Scheduled, Pub/Sub and Task Queue functions
- Cold starts, minimum instances and concurrency
- Idempotency, retries and dead-lettering
- The trigger loop, and how to break it
- Secrets, environment configuration and the Admin SDK
- Structured logging, error reporting and emulator debugging

Module 8: Cloud Storage & Cloud Messaging (Week 4)

Files and notifications, the two features almost every mobile brief includes. Cloud Storage covers upload and download with resumable transfers and progress, path design that lets a rule authorise by ownership, metadata and content types, signed URLs, and image processing triggered on upload so that thumbnails are generated server-side rather than on the device.

Cloud Messaging then covers the delivery path properly: device tokens and their rotation, topic and condition targeting, notification against data messages and how each behaves in foreground, background and terminated states on both Android and iOS, APNs configuration, deep links into the app, and delivery reporting. Rate limits, batching and the discipline of not sending a notification nobody asked for close the module.

- Resumable uploads, downloads and progress reporting
- Storage path design that rules can authorise
- Metadata, content types and signed URLs
- Image resizing triggered on upload
- FCM device tokens and token rotation
- Topic and condition targeting
- Notification against data messages across app states
- APNs setup, deep links and delivery reporting
- Rate limits, batching and notification restraint

Module 9: Hosting, App Hosting & Web Deployment (Week 5)

Getting the web half of the product in front of users. Firebase Hosting for static and single-page applications — the global CDN, cache headers, rewrites and redirects, custom domains and automatic certificates, preview channels for pull-request review, and multi-site hosting from one project.

App Hosting is then used for framework-driven applications that need server rendering, with Next.js and Angular as the worked examples: the GitHub-connected build and deploy flow, server-side rendering against Cloud Run, environment configuration and secrets, and rollbacks. The module ends on release discipline — promoting the same build through staging to production rather than rebuilding, and the deploy pipeline in GitHub Actions that automates it.

- Firebase Hosting, the CDN and cache headers
- Rewrites, redirects and single-page routing
- Custom domains and automatic certificates
- Preview channels for pull-request review
- Multi-site hosting from one project
- App Hosting for Next.js and Angular
- Server-side rendering on Cloud Run
- Environment configuration, secrets and rollbacks
- Deploy pipelines in GitHub Actions

Module 10: Analytics, Crashlytics, Remote Config & A/B Testing (Week 5)

The instrumentation that separates a shipped app from a maintained one. Google Analytics for Firebase covers automatic and custom events, event parameters and user properties, audiences, funnels and retention, the DebugView used to verify an event before release, and the BigQuery export that makes the raw event data queryable in SQL.

Crashlytics follows — crash and non-fatal reporting, symbol upload for readable stack traces, custom keys and logs that make a crash reproducible, and velocity alerts. Remote Config then turns configuration into a server-side decision with conditions, percentage rollouts and a kill switch for a feature that is misbehaving, and A/B Testing builds on both to run a real experiment with a stated metric. Performance Monitoring and App Check close the module.

- Automatic and custom events, parameters and user properties
- Audiences, funnels, retention and DebugView
- The BigQuery export and SQL over raw events
- Crashlytics, symbol upload and readable stack traces
- Custom keys, logs and velocity alerts
- Remote Config conditions and percentage rollouts
- Feature flags and a kill switch
- A/B testing with a stated success metric
- Performance Monitoring and App Check

Module 11: Firebase Genkit, Gemini & Data Connect (Week 6)

The two additions that changed what a Firebase backend can be asked to do. Genkit is the framework for AI features: flows as testable units, Gemini model calls, structured output with a schema so the response can be trusted by the calling code, retrieval-augmented generation over your own Firestore data, tool calling, and the developer UI for tracing and evaluating a flow. Flows are deployed as Cloud Functions and rate-limited, with cost and prompt-injection risk treated as engineering concerns rather than footnotes.

Data Connect then covers the case Firestore is wrong for: a managed PostgreSQL database inside a Firebase project, with a schema, GraphQL-shaped queries and mutations, generated typed SDKs, and authorisation expressed alongside them. The judgement is the point — relational integrity, joins and reporting belong in Data Connect; documents, real-time and offline belong in Firestore, and a real application often uses both.

- Genkit flows, and testing them as units
- Gemini model calls and structured output schemas
- Retrieval-augmented generation over Firestore data
- Tool calling and the Genkit developer UI
- Deploying flows as Cloud Functions, with rate limits
- Token cost, latency and prompt-injection risk
- Data Connect — managed PostgreSQL in a Firebase project
- Schema, queries, mutations and generated typed SDKs
- Choosing between Firestore and Data Connect

Module 12: Cost, Performance & Production Readiness (Week 6)

The module that stops a working prototype from becoming an expensive incident. Firebase bills per operation, so cost is a design property: a listener attached to a large collection, an unbounded query, a counter updated on every page view, or a trigger that writes back into its own collection are each traced from code to line item, with budget alerts and quota limits configured as a backstop.

Production readiness is then run as a checklist against each student's own project: security rules tested rather than assumed, App Check enforced so the backend only answers the real app, indexes declared in source control, backups and Firestore point-in-time recovery, IAM tightened so nobody develops with owner rights, monitoring and alerting, and a documented incident path. Vendor lock-in and an honest exit strategy close the module.

- Reading the bill — reads, writes, storage and egress
- The query and listener patterns that cost the most
- Budget alerts, quotas and a spending backstop
- App Check enforcement across services
- Indexes and rules kept in source control
- Backups and Firestore point-in-time recovery
- IAM least privilege for a team
- Monitoring, alerting and an incident path
- Vendor lock-in and a realistic exit strategy

Module 13: Capstone Project & Interview Preparation (Week 7 + placement prep)

The capstone is a complete product on Firebase: authenticated users, a Firestore model designed for its read patterns, security rules with a test suite, Cloud Functions for the logic that cannot live on the client, file upload with server-side processing, push notifications, one Genkit AI feature, analytics and Crashlytics wired in, and a web client deployed on Hosting or App Hosting — released through a pipeline with a staging project in front of production.

Interview preparation targets what Pune panels ask a Firebase candidate, which is rarely trivia: model this feature in Firestore and justify the reads it costs, write a security rule for this requirement, explain what happens to this write offline, and say where Firebase would be the wrong choice. Resume, LinkedIn and GitHub are rewritten around the capstone, and two mock interviews are run and reviewed.

- Capstone — auth, Firestore model, rules and Functions
- File upload, notifications and one Genkit AI feature
- Analytics, Crashlytics and App Check in place

- Web client deployed through staging to production
- A tested rules suite and a documented data model
- Modelling and rules questions under interview conditions
- Explaining offline behaviour and cost trade-offs
- Where Firebase is the wrong choice, argued honestly
- Resume, LinkedIn and GitHub rewritten around the capstone

Module 14: AI-Assisted Development Workflow (Final Week)

The skill every 2026 hiring panel now probes for — building real work with AI in the loop, responsibly. Learn to drive AI assistants (GitHub Copilot, Claude, Cursor, and IDE-native AI) to scaffold and accelerate the tools and stack this course covers, generate tests, explain and refactor unfamiliar code, and cut the boilerplate — while keeping you firmly in control of every decision. Heavy focus on guardrails: reviewing each AI suggestion, spotting hallucinated APIs or wrong answers, and handling licensing and data-privacy concerns. Close with a mini-project that takes a deliverable end-to-end using an AI-assisted workflow, then fold the same tooling into version control and everyday team practice.

- AI assistants — GitHub Copilot, Claude, Cursor, IDE-native AI
- Effective prompting for this course's stack — scaffolding, boilerplate, config
- AI-assisted test generation and coverage
- Explaining, refactoring, and modernising unfamiliar code with AI
- AI-driven review, error detection, and quality checks
- Generating and maintaining documentation with AI
- AI debugging — interpreting errors, logs, and failing output
- Guardrails — reviewing output, avoiding hallucinations, licensing & data privacy
- Team workflow — AI in the editor, in reviews, and in delivery pipelines
- Mini-project — a deliverable built end-to-end with an AI-assisted workflow

Capstone Projects

Full-Stack Mobile App on Firebase

A complete mobile app with Firebase as the entire backend — pick a real domain (chat, expense tracker, recipe finder, fitness tracker). Firebase Auth (email / Google / Apple), Firestore for data with proper security rules, Cloud Functions for server-side logic (order processing, scheduled cleanups), Cloud Storage for file uploads, FCM for push notifications, plus Crashlytics and Analytics. Frontend can be Android / iOS / Flutter / React / React Native — whichever you've trained on. Outcome: a deployed mobile app + GitHub repo demonstrating the Firebase patterns that Pune startup mobile teams hire on.

- Technology: Firebase Auth + Firestore + Cloud Functions
- Technology: Cloud Storage + FCM
- Technology: Crashlytics + Analytics
- Technology: Frontend: Android / iOS / Flutter / React / RN

Genkit-Powered AI Feature on Firebase

A 2026-relevant capstone — add a Gemini-powered AI feature to a Firebase app via Firebase Genkit (chat assistant, document Q&A, content recommendation, image-description). Demonstrates the modern Firebase + AI integration pattern.

- Technology: Firebase Genkit
- Technology: Gemini via Vertex AI
- Technology: Firestore for conversation persistence
- Technology: Cloud Functions for the orchestration

Career Outcomes

Firebase fluency is increasingly expected on Pune mobile-developer / full-stack JDs — Indeed Pune doesn't list 'Firebase Developer' as a standalone title (it's not), but Firebase appears as required or preferred on 800+ Pune mobile / full-stack JDs. The biggest employers using Firebase are Pune-based fintech / consumer-tech / health-tech / ed-tech startups, plus

several captives and BFSI mobile teams.

Firestore is most valuable as a force-multiplier on existing mobile / full-stack roles — adding Firestore to your Android / iOS / Flutter / React profile makes you immediately useful at a startup or solo-developer setting. Compensation is similar to your underlying mobile / full-stack role; Firestore fluency doesn't itself create a separate salary band.

The course doubles as an excellent foundation for our follow-on AI Engineer / Generative AI tracks — many AI-powered Pune mobile features are built on Firestore + Gemini.

Roles after this course

- Mobile Developer with Firestore fluency
- Full Stack Developer (Firestore backend)
- Junior Solutions Engineer at startups
- Solopreneur / freelancer shipping mobile apps
- Junior AI Engineer (with Genkit)

Indicative salary bands

- Mobile Developer with Firestore fluency (Pune entry, <2 years) - Rs. 4,00,000 – Rs. 7,00,000 per year (source: AmbitionBox Pune Mobile Developer)
- Mid-level Full Stack with Firestore (Pune, 3–5 years) - Rs. 10,00,000 – Rs. 17,00,000 per year (source: Glassdoor Pune Full Stack Developer)
- Senior Mobile / Full Stack with Firestore (Pune, 5–8 years) - Rs. 16,00,000 – Rs. 26,00,000 per year (source: Glassdoor Pune Senior Full Stack)

Companies hiring for these skills

- Pune-based fintech / consumer-tech / health-tech / ed-tech startups
- Pune solo-developer / freelance ecosystem
- BharatPe (Pune)
- Razorpay (Pune)
- Pine Labs (Pune)
- Drip Capital
- Innovaccer (Pune)
- Whatfix (Pune)
- BFSI mobile teams (specific apps)

Batch Modes and Duration

Duration: 6 weeks of structured curriculum plus 1 week of capstone (~1.5 months total)

Classroom

- Location: Archer Infotech, Kothrud, Pune
- Morning batch — 10:00 to 13:00
- Evening batch — 18:00 to 21:00

Live online

- Same hours as classroom batches
- Recordings available for review
- Zoom for live sessions
- Firestore Console + emulator suite
- Firestore CLI
- GitHub for capstone
- Slack / WhatsApp for async Q&A

Weekend

- Saturday + Sunday, 09:00 to 13:00
- Stretches over ~2.5 months instead of 1.5.

Maximum 15 students per batch.

Fees

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession — Firestore as a 1.5-month course typically lands at the lower end. Firestore Spark (free tier) covers all lab work; Blaze (pay-as-you-go) is needed only for Cloud Functions usage above the free quota.

Indicative range: Rs. 20,000 – Rs. 90,000

- Single payment with early-bird discount
- EMI in 2 instalments at no extra cost
- Corporate sponsorship — invoiced with GST

Placement Support

Placement focus for Firestore students is calibrated for Mobile / Full Stack roles where Firestore is a complement skill (not a standalone hireable specialisation). Our support is unconditional, time-bound (six months after course completion), and includes free re-entry to a future batch's interview-prep sessions.

The placement process

- Resume + LinkedIn rewrite emphasising Firestore fluency on top of mobile / full-stack background
- GitHub portfolio cleanup
- Mock interview rounds for Mobile / Full Stack roles
- Post-course referrals via our 17-year alumni network (with extra emphasis on Pune startups)
- Up to 6 months of continued support
- Free re-entry to future batch interview-prep sessions

Partner companies

- Pune-based fintech / consumer-tech / health-tech / ed-tech startups
- BharatPe (Pune)
- Razorpay (Pune)
- Pine Labs (Pune)
- Drip Capital
- Innovaccer (Pune)
- Whatfix (Pune)

Prerequisites and How to Start

Prerequisites: at least basic JavaScript or another programming language, basic familiarity with mobile or web development (you should have done at least one frontend track — Android, iOS, Flutter, React, RN, or web JS — before this course).

Willingness to commit 5–6 hours per week of practice.

Suggested first steps

- Decide your mode — classroom, online live, or weekend
- Check the upcoming batch dates
- Book a free 30-minute counselling call
- Confirm enrolment and complete pre-course orientation (Firestore account creation, Firestore CLI install)
- Show up to day one with a laptop running 64-bit OS

Frequently Asked Questions

How long does Firestore training in Pune take at Archer Infotech?

Approximately 1.5 months — 6 weeks plus 1 week of capstone. Weekend batch ~2.5 months.

Is Firestore free?

Firestore Spark is the free tier (generous quotas for development and small apps). Blaze is pay-as-you-go for production. The course lab work runs entirely on Spark for most students.

What is the salary impact?

Firestore fluency adds value to existing Mobile / Full Stack roles rather than creating a separate salary band. Junior mobile developers with Firestore Rs. 4–7 lakh; mid-level full stack with Firestore Rs. 10–17 lakh; senior Rs. 16–26 lakh.

Do I need a mobile or web background?

Yes — Firestore is a backend complement to a frontend stack. You should have done at least one frontend track (Android, iOS, Flutter, React, RN, or web JS) before this course.

Will I work on real projects?

Yes — two capstone projects: (1) full-stack mobile app entirely on Firestore, (2) Genkit-powered AI feature on Firestore using Gemini.

Is Genkit / GenAI covered?

Yes — week 6 covers Firestore Genkit and Gemini integration. Capstone Project #2 is a complete Genkit-powered feature.

Are weekend Firestore classes available in Pune?

Yes — Saturday and Sunday, 09:00–13:00, stretched over ~2.5 months instead of 1.5.

What is the fee?

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession.

What support do I get after course completion?

Six months of placement support, referrals via our alumni network (extra emphasis on Pune startup scene), mock interviews.

Are the named trainers actually teaching?

Amol Patil and Amol Chougule personally lead every session.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.